

Demo

Hệ thống Multi-Agent System hỗ trợ research, tổng hợp, phân tích thông tin và viết báo cáo từ nhiều nguồn.

Tech Stack (đề xuất)

Frontend: React + TailwindCSS + TypeScript (đa số đồ án cũ toàn dùng cái này, đợt này dùng Typescript sẽ thay cho Javascript để đảm bảo type safety và dễ maintain hơn - no js, bug runtime khổ lắm :v)

Muốn mới hơn thì dùng Next.js cũng dc, hoặc là dùng streamlit của python luôn.

Backend: FastAPI (Python, vì các multi-agent này được build trên LLM, python support tốt nhất nên dùng python luôn, mindset backend từ js/ts (Express hay NestJS) mang qua đây cũng ko đổi nhiều lắm, học thêm tí syntax FastAPI)

Riêng demo này thì ko dùng rest api để nhận request response như mỗi lần mà sẽ dùng websocket để hỗ trợ streaming response luôn.

Dev python nhớ dùng venv + requirements.txt để quản lý deps của python nha, đừng có cài trực tiếp lên máy, conflict version với máy nữa thì khổ. (kiểu như npm/yarn/bun với cái node_modules bên js/ts ấy, python thì dùng venv + requirements.txt)

```
python3 -m venv .venv
source .venv/bin/activate
```

LLM: Gemini API (Google) - api key free, dùng model gemini-3.5-flash-lite (model với tốc độ response khá nhanh, 15 req/minute, 500 req/day), ngoài ra sẽ kẹp thêm các model khác để fallback xuống nếu bị rate limit 429, 503, ... or something error...

<https://aistudio.google.com/api-keys>

Search engine: Tavily API - api key free (1000 req/month), dùng để search thông tin từ web, hoặc kẹp thêm DuckDuckGo (DuckDuckGo thì ko cần api key, call api trực tiếp luôn)

<https://app.tavily.com/playground>

Tools:

- LangChain (Python) - framework cung cấp abstractions cho LLM, tools, structured output...
- **LangGraph (Python)** - cùng hệ sinh thái với LangChain, dùng riêng cho phần **điều phối luồng giữa 4 agent** (state, conditional routing, streaming event). Chi tiết ở phần "Orchestration engine" bên dưới.
- Dockerfile + docker-compose + ci/cd pipeline: build, test, deploy image tự động rồi chạy trong container để đảm bảo environment dev/prod.
- Logging: với frontend (typescript + react), dùng pino, backend python thì dùng loguru (nhớ logging để còn biết lỗi là gì)

Kiến trúc

Kiến trúc được thiết kế theo mô hình **pipeline multi-agent** gồm 4 agent chính. Mỗi agent có một vai trò riêng, nhận input có cấu trúc từ agent trước đó và trả output cho agent tiếp theo.

Các tầng trong kiến trúc

1. Orchestration agent

Agent trung tâm tiếp nhận yêu cầu, phân tích mục tiêu, chia nhỏ task và điều phối ba agent còn lại. Nó quản lý workflow từ đầu đến cuối, kiểm tra kết quả trung gian và yêu cầu làm lại hoặc research bổ sung khi cần.

2. Researcher agent

Researcher tìm kiếm, thu thập và trích xuất thông tin từ nhiều nguồn như search engine, API, tài liệu nội bộ và website. Mỗi finding cần đi kèm nguồn, URL và dữ liệu liên quan; agent không tự viết báo cáo cuối.

3. Analyst agent

Analyst nhận findings từ Researcher để chuẩn hóa, so sánh và phân tích. Agent phát hiện mâu thuẫn giữa các nguồn, đánh giá độ tin cậy, rút ra kết luận và insight. Nếu dữ liệu thiếu hoặc chưa đủ tin cậy, Analyst phản hồi cho Orchestration để yêu cầu Researcher bổ sung.

4. Writer agent

Writer chuyển kết quả đã được Analyst kiểm tra thành deliverable cuối: báo cáo, tóm tắt, câu trả lời, outline hoặc nội dung theo template. Writer không tự ý bổ sung số liệu chưa được kiểm chứng và phải giữ lại các citation, cảnh báo hoặc giới hạn của dữ liệu.

Orchestration engine:

- **LangChain:** nằm *bên trong* từng agent. Vai trò của nó là giúp 1 agent đơn lẻ "làm việc" được với LLM — gọi Gemini, gọi tool search (Tavily/DuckDuckGo), và ép output của model trả về đúng format JSON theo schema đã định nghĩa ở trên, thay vì để model trả lời tự do rồi phải tự parse tay.
- **LangGraph:** nằm *bên ngoài*, bao quanh cả 4 agent. Vai trò của nó là quyết định **cả pipeline chạy như thế nào** — agent nào chạy trước, agent nào chạy sau, khi nào cần quay lại Researcher, khi nào dừng hẳn. Nói dễ hiểu thì LangGraph đóng vai trò thay cho phần logic "Orchestration agent" đang được mô tả bằng chữ ở trên, giờ được triển khai thật bằng framework thay vì tự viết if/else tay.

Cách LangGraph mô hình hóa hệ thống: mỗi agent (Orchestration, Researcher, Analyst, Writer) được coi là 1 **node**. Đường đi giữa các node gọi là **edge**. Có 2 loại edge:

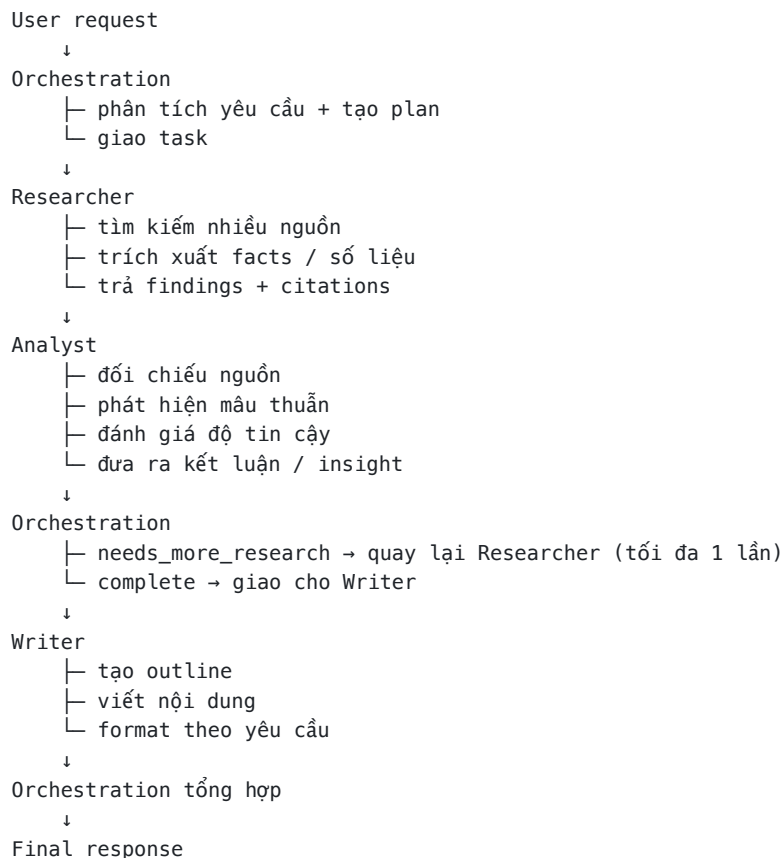
- Edge thường: Orchestration → Researcher → Analyst, luôn đi theo 1 chiều cố định.
- **Conditional edge** (edge có điều kiện): sau khi Analyst chạy xong, LangGraph tự kiểm tra field `status` trong output — nếu là `needs_more_research` thì tự động quay lại Researcher, nếu là `complete` thì đi tiếp tới Writer. Đây chính là phần thể hiện đúng sơ đồ feedback loop đã vẽ ở mục "Feedback loop và điều kiện dừng".

Ngoài ra LangGraph còn quản lý giúp 2 việc:

- **State dùng chung:** tương đương với "context chung của phiên làm việc" mà Orchestration đang giữ — LangGraph tự lo việc truyền dữ liệu (plan, findings, analysis...) giữa các node, không cần tự viết code pass tay qua từng hàm.
- **Streaming event:** mỗi khi 1 node (agent) chạy xong, LangGraph có thể tự bắn ra 1 sự kiện — đây chính là nguồn để đẩy các `agent_status` event qua WebSocket cho frontend hiển thị progress real-time, thay vì phải tự cắm log thủ công ở từng chỗ gọi agent.

Tóm gọn: **LangGraph gọi lần lượt các agent theo đúng luồng (kể cả luồng có lặp lại), còn bên trong mỗi agent vẫn dùng LangChain để thực sự gọi LLM và tool.** Giới hạn "Researcher chỉ được chạy bổ sung tối đa 1 lần" cũng được implement ngay trong phần điều kiện routing này của LangGraph, chứ không cần code kiểm tra riêng ở chỗ khác.

Luồng dữ liệu chi tiết



Orchestration xuất hiện 2 lần trong luồng: 1 lần sau Analyst (quyết định đi Writer hay quay lại Researcher), 1 lần sau Writer (tổng hợp/format trước khi trả về). Toàn bộ output của Analyst và Writer đều phải đi qua Orchestration trước, đúng như 2 schema Analyst → Orchestration và Writer → Orchestration ở phần dưới — không có agent nào gọi thẳng agent khác mà bỏ qua Orchestration.

State và cách các node giao tiếp

Orchestration giữ context chung của phiên làm việc và truyền cho mỗi agent đúng phần thông tin cần thiết. Researcher trả về findings và nguồn; Analyst trả về bản phân tích có kết luận, mức độ tin cậy và các điểm chưa chắc chắn; Writer dùng bản phân tích đó để tạo nội dung cuối. Nếu Analyst phát hiện thiếu dữ liệu, workflow có thể quay lại Researcher thông qua Orchestration.

Với LangGraph, phần "context chung" này chính là state dùng chung giữa các node đã nói ở mục "Orchestration engine" — LangGraph tự lo việc truyền state giữa các node, không cần tự pass tay qua từng hàm.

Schema output giữa các agent

Các agent không nên giao tiếp bằng đoạn text tự do hoàn toàn. Mỗi agent trả về output có cấu trúc để agent tiếp theo dễ xử lý và để frontend có thể hiển thị tiến trình.

Researcher → Analyst:

```
{
  "status": "complete",
  "findings": [
    {
      "claim": "Nội dung hoặc số liệu được tìm thấy",
      "evidence": "Đoạn trích hoặc dữ liệu liên quan",
      "source_url": "https://example.com/source",
      "source_title": "Tên nguồn",
      "published_at": "2025-01-01"
    }
  ],
  "search_queries": ["query đã sử dụng"],
  "limitations": []
}
```

Analyst → Orchestration:

```
{
  "status": "complete",
  "confidence": 0.85,
  "verified_findings": [],
  "conclusions": [],
  "insights": [],
  "conflicts": [],
  "limitations": [],
  "follow_up_request": null
}
```

Khi cần tìm thêm dữ liệu, Analyst trả về:

```
{
  "status": "needs_more_research",
  "confidence": 0.55,
  "conflicts": ["Hai nguồn đưa ra số liệu khác nhau"],
  "follow_up_request": {
    "questions": ["Xác minh số liệu bằng nguồn chính thức"],
    "preferred_sources": ["official report", "government database"]
  }
}
```

Writer → Orchestration:

```
{
  "status": "complete",
  "title": "Tiêu đề deliverable",
  "content": "Nội dung cuối cùng",
  "citations": [],
  "warnings": []
}
```

Feedback loop và điều kiện dừng

Feedback loop chỉ được điều phối thông qua Orchestration; Analyst không gọi trực tiếp Researcher. Luồng mặc định là:

```
Researcher → Analyst → Orchestration → Writer
                        |
                        └─ needs_more_research → Researcher
```

Để phù hợp với bản demo, mỗi request chỉ cho phép Researcher chạy bổ sung tối đa **1 lần**. Sau lần bổ sung đó:

- Nếu Analyst đánh giá dữ liệu đủ tin cậy, chuyển kết quả cho Writer.
- Nếu dữ liệu vẫn chưa đủ, Writer vẫn tạo output nhưng phải đưa cảnh báo vào `warnings` và nêu rõ giới hạn của kết quả.
- Nếu tool search hoặc nguồn ngoài bị lỗi, workflow trả trạng thái lỗi có kiểm soát thay vì lặp vô hạn.

Orchestration cũng cần có timeout và trạng thái kết thúc rõ ràng: `complete`, `needs_more_research`, `partial` hoặc `failed`.

Toàn bộ rule này (giới hạn 1 lần retry, route theo status) được implement trực tiếp trong phần conditional edge của LangGraph ở mục "Orchestration engine" bên trên.

Progress và status events

Frontend chỉ hiển thị các event trạng thái và hoạt động của agent, không hiển thị chain-of-thought hoặc suy luận nội bộ của model. Một event có thể có dạng:

```
{
  "type": "agent_status",
  "agent": "researcher",
  "status": "running",
  "message": "Đang tìm kiếm các nguồn liên quan",
  "timestamp": "2025-01-01T12:00:00Z"
}
```

Các trạng thái chính:

```
queued → running → completed
                ↳ failed
```

Ví dụ event cho một request:

```
Orchestration: Đã tạo research plan
Researcher: Đang tìm kiếm nguồn
Researcher: Đã thu thập 5 findings
Analyst: Đang đối chiếu dữ liệu
Analyst: Phát hiện mâu thuẫn, yêu cầu research bổ sung
Researcher: Đang xác minh lại nguồn chính thức
Writer: Đang tạo báo cáo
Orchestration: Hoàn tất
```

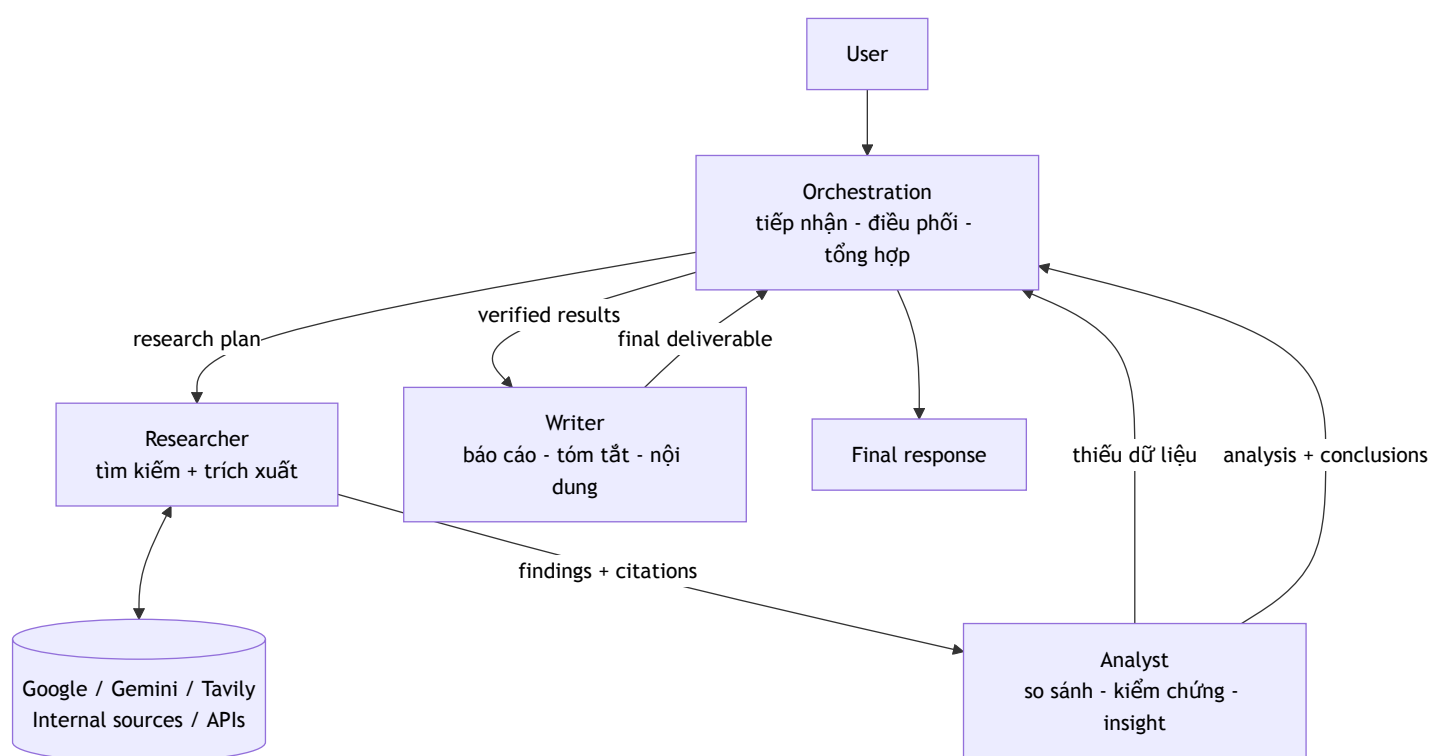
Ưu điểm

- **Phân tách trách nhiệm rõ:** mỗi agent tập trung vào một loại công việc, prompt và output dễ kiểm soát hơn.
- **Chất lượng tốt hơn một agent đơn:** kết quả được research, kiểm tra và viết lại qua nhiều bước thay vì một agent vừa tìm vừa kết luận vừa trình bày.
- **Có bước kiểm chứng độc lập:** Analyst giúp giảm hallucination, phát hiện mâu thuẫn và ngăn Writer dùng số liệu chưa đáng tin.
- **Dễ mở rộng:** có thể thay search tool của Researcher hoặc thêm chuyên gia domain mà không phải viết lại toàn bộ workflow.
- **Dễ quan sát và debug:** log output của từng agent, thời gian chạy, nguồn đã dùng và lý do Orchestration yêu cầu chạy lại.
- **Phù hợp nhiều loại output:** Writer có thể tạo báo cáo, câu trả lời, bảng so sánh, outline hoặc nội dung theo template.
- **Đỡ tự viết tay phần điều phối:** dùng LangGraph thì state, conditional routing và streaming event đã có sẵn pattern, không phải tự implement từ đầu.

Nhược điểm và trade-off

- **Tốn latency và token:** một request phải qua nhiều agent, đặc biệt khi Orchestration yêu cầu research bổ sung.
- **Chi phí vận hành cao hơn:** mỗi agent có thể gọi LLM và Researcher còn gọi thêm các dịch vụ search/API bên ngoài.
- **Phụ thuộc chất lượng handoff:** findings lộn xộn hoặc thiếu nguồn sẽ làm Analyst và Writer khó tạo kết quả tốt.
- **Có thể bị lặp workflow:** cần giới hạn số vòng, timeout và điều kiện dừng để Orchestration không yêu cầu research lại vô hạn.
- **Khó debug hơn agent đơn:** lỗi có thể nằm ở routing, prompt, tool, dữ liệu trung gian hoặc bước format cuối.
- **Không phải task nào cũng cần 4 agent:** câu hỏi đơn giản sẽ bị xử lý quá mức, chậm hơn và tốn chi phí không cần thiết.
- **Chất lượng nguồn bên ngoài không ổn định:** website thay đổi, bị chặn hoặc trả dữ liệu thiếu sẽ ảnh hưởng trực tiếp đến Researcher.
- **Thêm 1 dependency mới (LangGraph):** cần học thêm concept StateGraph /node/edge, tốn thời gian ban đầu nhưng phù hợp scope đồ án nên đáng để làm.

Mỗi agent sẽ có 1 prompt riêng, context cụ thể, role, task và output dự kiến của chúng để đảm bảo hoạt động đúng chức năng.



Flow

Khi người dùng nhập một truy vấn dạng:

Research the top 3 highest-grossing video games of all time as of 2025, with sales or revenue numbers, as a short r

Từ frontend --> gửi xuống phía backend --> orchestration agent sẽ nhận truy vấn, phân công nhiệm vụ cho researcher --> researcher tìm kiếm thông tin từ nhiều nguồn (Google, Gemini, Tavily, v.v.) và tổng hợp lại (search song song để tối ưu thời gian) --> analyst phân tích dữ liệu từ researcher, đưa ra kết luận, insight, phân tích --> orchestration nhận kết quả từ analyst, kiểm tra đủ dữ liệu chưa: nếu chưa đủ thì giao lại cho researcher tìm bổ sung (tối đa 1 lần), nếu đủ thì giao cho writer --> writer viết báo cáo, tóm tắt, hoặc tạo nội dung dựa trên kết quả đã kiểm tra --> orchestration nhận lại kết quả từ writer, tổng hợp và format lần cuối --> trả về frontend.

Ngoài ra sau khi người dùng gửi yêu cầu, bên dưới yêu cầu vừa gửi sẽ hiển thị progress/status events của từng agent để

người dùng theo dõi tiến trình xử lý theo thời gian thực.

Cấu trúc thư mục đề xuất:

Frontend thì theo react (components + hooks, ...), còn Backend thì theo FastAPI (agents tách rõ ràng từng con, tools tách riêng, ...)

```
multi-agent-research/
├── frontend/
│   ├── public/
│   │   └── ...
│   ├── src/
│   │   ├── components/
│   │   │   ├── chat/
│   │   │   │   ├── ChatInput.tsx
│   │   │   │   ├── ChatMessage.tsx
│   │   │   │   ├── ChatContainer.tsx
│   │   │   │   └── StreamingMessage.tsx
│   │   │   ├── research/
│   │   │   │   ├── ResearchProgress.tsx
│   │   │   │   ├── SourceList.tsx
│   │   │   │   └── ResearchReport.tsx
│   │   │   └── ui/
│   │   │       ├── Button.tsx
│   │   │       ├── Card.tsx
│   │   │       └── ...
│   │   ├── hooks/
│   │   │   └── useResearchSocket.ts
│   │   ├── services/
│   │   │   └── websocket.ts
│   │   ├── types/
│   │   │   ├── research.ts
│   │   │   └── websocket.ts
│   │   ├── App.tsx
│   │   ├── main.tsx
│   │   └── index.css
│   ├── Dockerfile                # build image frontend cho demo
│   ├── package.json
│   ├── tsconfig.json
│   ├── vite.config.ts
│   └── ...
├── backend/
│   ├── app/
│   │   ├── agents/
│   │   │   ├── base.py           # interface chung: run(state) + output Pydantic
│   │   │   ├── orchestration/
│   │   │   │   ├── agent.py      # planner/synthesizer LLM, không chứa graph
│   │   │   │   └── prompts.py
│   │   │   ├── researcher/
│   │   │   │   ├── agent.py
│   │   │   │   └── prompts.py
│   │   │   ├── analyst/
│   │   │   │   ├── agent.py
│   │   │   │   └── prompts.py
│   │   │   └── writer/
│   │   │       ├── agent.py
│   │   │       └── prompts.py
│   │   ├── graph/
│   │   │   ├── state.py          # shared state: plan, findings, analysis...
│   │   │   └── build.py          # định nghĩa StateGraph (node, edge)
```

```
├── routing.py          # conditional edge: needs_more_research, max 1 retry
├── tools/
│   ├── search/
│   │   ├── tavily.py
│   │   └── duckduckgo.py
│   ├── llm/
│   │   ├── gemini.py
│   │   └── factory.py  # chọn model + fallback khi 429/5xx/timeout
│   └── ...
├── services/
│   ├── research_service.py  # gọi graph để chạy pipeline
│   └── websocket_service.py
├── schemas/
│   ├── research.py
│   └── websocket.py
├── core/
│   ├── config.py
│   └── logging.py
├── api/
│   └── websocket.py
├── main.py
├── tests/
│   ├── agents/
│   ├── tools/
│   └── ...
├── Dockerfile          # build image backend cho demo
├── .env
├── .env.example
├── requirements.txt
├── ...
├── .gitignore
├── README.md
└── docker-compose.yml  # dựng chung frontend + backend
```